# Boucles et organigrammes

## Cours et exemples

Maths Explorer

# Objectifs

Exécuter et construire une boucle, produire un organigramme et justifier la terminaison d'un algorithme simple.

## Exécuter sans sauter une instruction

Une affectation $a \leftarrow E$ remplace la valeur de $a$ par la valeur actuelle de $E$. Elle ne constitue pas une égalité à résoudre. Une boucle « pour » parcourt une liste d'indices ; une boucle « tant que » répète tant que sa condition est vraie. Le test de « tant que » précède chaque tour, y compris le premier.

# Exécuter sans sauter une instruction

Lire un entier $n \geq 0$
$S \leftarrow 0$
Pour $k$ allant de 1 à $n$
$S \leftarrow S + k$
Fin pour
Afficher $S$

## Exécuter sans sauter une instruction (2)

Pour $n = 4$, les valeurs successives de $S$ sont $0, 1, 3, 6, 10$. Si $n = 0$, aucun tour n'est effectué et la somme vide vaut 0.

## Exécuter sans sauter une instruction

Après le tour d'indice $k$, $S = 1 + \cdots + k$.

## Exécuter sans sauter une instruction

Avant le premier tour, $S = 0$. Si la somme est correcte au tour précédent, l'ajout de $k$ donne celle des $k$ premiers entiers. Une récurrence justifie chaque tour ; après $n$ tours, le résultat annoncé est obtenu.

## Boucle conditionnelle et terminaison

Avec $u \leftarrow 1$, $n \leftarrow 0$, tant que $u < 20$, faire $u \leftarrow 2u$ puis $n \leftarrow n + 1$. Les valeurs de $u$ sont $1, 2, 4, 8, 16, 32$ ; la sortie est $n = 5$. Après $n$ tours, $u = 2^n$. La suite des puissances dépasse 20 : l'arrêt est justifié.

## Boucle conditionnelle et terminaison

Si le corps ne modifie pas ce qui intervient dans le test, la boucle peut être infinie. Pour permuter deux valeurs, $a \leftarrow b$ puis $b \leftarrow a$ ne convient pas : la première valeur de $a$ a été perdue. Utiliser une variable temporaire.

## Boucle conditionnelle et terminaison

Préciser les entrées autorisées ; dresser une table d'exécution ; expliquer ce qui reste vrai après chaque tour (invariant) ; expliquer pourquoi le test finit par être faux (terminaison) ; interpréter la sortie et les cas limites.

## Boucle conditionnelle et terminaison

On initialise $a = 7, b = 3$, puis on exécute $a \leftarrow a - b$ et $b \leftarrow a + b$.

| État | $a$ | $b$ |
|---|---|---|
| Initial | 7 | 3 |
| Après la première affectation | 4 | 3 |
| Après la seconde | 4 | 7 |

## Boucle conditionnelle et terminaison (2)

Dans la seconde ligne de calcul, $a$ vaut déjà 4. Remplacer les deux variables simultanément par leurs anciennes valeurs décrirait un autre algorithme.

## Boucle conditionnelle et terminaison

Un invariant explique ce que contiennent les variables si l'algorithme atteint une étape. Il ne garantit pas que l'algorithme s'arrête. Pour une boucle bornée, compter les tours ; pour une boucle conditionnelle, trouver une quantité qui finit par franchir le seuil. Tester aussi le cas où la condition est fausse dès le départ.

## Boucle conditionnelle et terminaison

Pour calculer $u_0 + \cdots + u_N$ quand $u_{n+1} = 2u_n + 1$, initialiser $u = u_0, S = u_0$. Au tour $k = 1, \dots, N$, mettre d'abord $u \leftarrow 2u + 1$, puis $S \leftarrow S + u$. Après le tour $k$, $u = u_k$ et $S = u_0 + \cdots + u_k$. En $N = 0$, il n'y a pas de tour et la sortie est déjà correcte. Inverser les deux mises à jour compterait deux fois $u_0$ et oublierait le dernier terme.

## Boucle conditionnelle et terminaison

Une suite peut tendre vers un seuil sans l'atteindre à un rang fini. Un test d'égalité exacte ne doit pas être remplacé tacitement par un test approché. Pour demander une précision, fixer une tolérance strictement positive dans l'énoncé.